Demo segment - full script and step-by-step explanation

Topic 8 - AI-Assisted DevOps - slides 27-34 - 7:00 - speaker: Huy

The demo is worth **20%** on its own, and it feeds another **20%** through Critical analysis - because the most valuable thing in it is the part where we say what we did *not* build.

This document has five parts.

A the honesty frame - what is real and what is not **B** the nine steps, each one explained: what the code does, why the step exists, what appears on screen, what to say, and what you will be asked **C** the second demo - the enforcement experiment **D** questions this demo invites, with answers **E** setup checklist and what to do when it breaks

---

Timing

	What	Time	Running
Slide 27	Two demos - and what is real	0:50	0:50
LIVE 1	`py triage_demo.py --focus 5,7 --step`	2:20	3:10
LIVE 2	`py debug_trace.py --mode prompt-only`	0:45	3:55
LIVE 3	`py debug_trace.py --mode tool-based`	0:45	4:40
Slide 31	It's a tie	0:45	5:25
Slide 32	Why tool-based anyway	0:50	6:15
Slide 33	Structure is not semantics	0:45	7:00

Slides 28 and 34 are **absorbed** - the live run shows the input log and the final comment better than a screenshot can. Skip past them. If you are running early, land on 34 for fifteen seconds at the end.

PART A - The honesty frame

Slide 27 *(0:50)*

SAY

> I'm going to show you two things. The whole workflow end to end, then the experiment behind one design decision.
>
> But first, thirty seconds of honesty, because you should know what you're looking at.
>
> Everything you're about to see is our real code. All nine classes from the architecture slide. Real HMAC signature checking. Real log trimming. A real call to the Claude API - we pay for it.

What we did *not* build is the network plumbing around it. There is no HTTP server listening. There is no public tunnel. We are not triggering a real CI run.

Instead we call the service directly with a saved event. That is the whole difference.

We removed the I/O we don't control. We kept every part we wrote.

Fifty seconds. Then move. The detail lives in Part D if anyone pushes.

The precise inventory - keep this in your head

Step	Class	Real?	Note
1	`WebhookPayload.from_dict`	real	parses the genuine `workflow_run` payload shape
2	`SignatureVerifier`	real	`hmac.new(secret, body, sha256)` + `compare_digest`
3	`EventFilter`	real	three conditions, evaluated
4	`GitHubClient.get_workflow_logs`	**simulated**	log generated, not downloaded
5	`LogTrimmer.trim`	real	10 regex patterns over 12,017 lines
6	`PromptBuilder.build`	real	
7	`ClaudeClient.complete`	real	real HTTPS, real billed tokens, forced `tool_choice`
8	`ResponseValidator`	real	all six rules
9	`MarkdownFormatter` + post	real	posts for real with `--post`, otherwise writes a file

Not implemented at all: the HTTP receiver, `HistoryStore`, `RepoManager`, the dashboard, and the `run_id` idempotency check. They are in the design document. They are not in this demo. Say so if asked.

PART B - The nine steps

Command - already typed, cursor waiting, not yet entered:

```
py triage_demo.py --focus 5,7 --step
```

`--step` waits for Enter between steps. You control the pace. Nine presses, roughly fifteen seconds each. Steps 5 and 7 expand into full detail; the other seven print one line.

Beat 0 - the banner *(0:15)*

Before anything runs, a three-layer map appears: presentation, application, infrastructure, with step 7 in red.

SAY

Before it starts - this is the architecture slide again. Presentation layer,

application layer, infrastructure. Nine steps. The one in red is step seven.

That is the only AI in the system.

Step 1 - `WebhookPayload.from_dict()` *(0:10)*

What the code does. GitHub sends a JSON body for the `workflow_run` event. The real payload has hundreds of fields. We pull out eight and build a DTO: `event`, `action`, `status`, `conclusion`, `run_id`, `repo`, `pr_number`, `head_sha`. The interesting ones are nested - `status` and `conclusion` live inside `workflow_run`, and the PR number is inside `workflow_run.pull_requests[0].number`.

Why the step exists. This is the only place in the system that knows GitHub's payload shape. Everything downstream sees our eight-field object. If GitHub renames a field tomorrow, one method changes and nothing else does. That is what a DTO is *for* - it is not just a data bag, it is a boundary.

On screen. One line: the event, the conclusion in red, the run id, the PR number.

SAY

GitHub says a workflow run finished. Run 8471023, conclusion failure, on pull

request one.

If asked - "why not just pass the raw dict around?" Because then every class downstream depends on GitHub's schema, and a payload change breaks the whole system instead of one method.

Step 2 - `SignatureVerifier.verify()` *(0:15)*

What the code does. GitHub computes `HMAC-SHA256(webhook_secret, raw_request_body)` and sends it in the header `X-Hub-Signature-256: sha256=<hex>`. We recompute the same HMAC over the same bytes with the same shared secret and compare the two hex strings.

Two details that matter and that a marker may probe:

It must be the raw body. Not a re-serialized dict. If you parse the JSON and dump it again, key order and whitespace change, the bytes change, and the hash will never match. The verifier takes `bytes`, deliberately.

We use `hmac.compare_digest`, not `==`. String equality in Python short-circuits at the first differing byte. That means the comparison takes measurably longer the more leading bytes you got right - so an attacker can recover a valid signature one byte at a time by timing the responses. `compare_digest` always examines every byte. This is a timing-attack countermeasure, and it is one line.

Why the step exists. The endpoint is on the public internet. Without this, anyone can POST a fabricated failure event. They would make the bot comment whatever they like on our pull requests and burn our API credits doing it. It is step two of nine on purpose - before we spend a single cent or read a single byte of log.

On screen. The received signature and the computed signature, one above the other, then the comparison result.

SAY

Signature check. Anyone on the internet can POST to that endpoint, so before

we do anything else we recompute the HMAC over the raw body and compare.

An unsigned request never reaches the model. This is step two of nine, on

purpose.

If asked - "the demo signs the body itself, so isn't the check trivially passing?" Yes, and that's fair. In the demo we generate the header the way GitHub would, because there is no GitHub. The *verification* code is real; the adversary is absent. To see it reject, change one byte of the secret and re-run.

Step 3 - `EventFilter.is_valid_failure_event()` *(0:10)*

What the code does. Three equality checks: `event == "workflow_run"`, `status == "completed"`, `conclusion == "failure"`. All three must hold.

Why three and not one. GitHub fires `workflow_run` three times per run - `requested`, `in_progress`, `completed`. And `completed` is not the same as *failed*: the conclusion can be `success`, `failure`, `cancelled`, `skipped`, `timed_out`, `action_required`, or `neutral`. Without both checks we would call a paid API on every green build, which is the overwhelming majority of traffic, and post triage comments on pull requests where nothing broke.

Why it returns 204, not an error. A filtered event is a normal event, not a fault. Returning an error code would make GitHub retry the delivery, and retrying something we deliberately ignored is a loop we do not want.

On screen. Three conditions with a tick each, showing expected against actual.

SAY

> Filter. GitHub sends us every run, including the green ones. Three
>
> conditions. We only pay for red.

Step 4 - `GitHubClient.get_workflow_logs()` *(0:15)*

What the real code does. `GET /repos/{owner}/{repo}/actions/runs/{run_id}/logs` returns a 302 redirect to a short-lived signed URL holding a ZIP of one log file per job. You follow the redirect, unzip, and concatenate.

What the demo does instead. Generates 12,017 lines from a seeded random number generator, with the genuine pytest failure appended at the end.

Why simulated. Two reasons, and give both:

Reproducibility. The same seed produces the same log every run. That means the trimming ratio we quote is a measurement, not an estimate, and the demo behaves identically in rehearsal and on stage.

It tests nothing of ours. Downloading a ZIP over HTTP is not where our design risk lives. Making it live would add a dependency on the venue network in exchange for no new information.

The numbers, and why they matter. 12,017 lines. 959 KB. Roughly 240,000 tokens. That is the number to say out loud, because it makes the next step inevitable rather than clever.

On screen. The first three lines and the last three lines of the log, then the size.

SAY

> And here's the log. Twelve thousand and seventeen lines. Nine hundred and
>
> fifty-nine kilobytes. That's roughly two hundred and forty thousand tokens.
>
> We cannot send that. Watch the next step - this is the one that matters.

Step 5 - `LogTrimmer.trim()` - expanded *(0:40)*

This is the best forty seconds of the demo. Slow down here.

What the code does, in order:

1. **Keep the tail unconditionally** - the last 40 line indices. The end of a run almost always carries the conclusion.
2. **Scan for error patterns.** Ten regexes compiled into one alternation: `##[error]`, `^E\s`, `\bFAILED\b`, `Traceback`, `\bERROR\b`, `fatal:`, `Exception`, `SyntaxError`, `was canceled`, `exceeded the maximum`. Every matching line index goes into the keep set.
3. **Add context.** For every match at index *i*, also keep *i−3* through *i+3*.
4. **Cap it.** Take at most the last 60 kept indices.
5. **Render with gap markers** - `... N lines omitted ...` wherever the kept indices are not consecutive.

Why each of those five choices exists - this is the part that shows design thinking, so be ready to give any of them:

Why keep the tail at all? Because some failures print their verdict only at the end. `1 failed, 127 passed` is the last line of a pytest run.

Why not tail only? Because the tail of a pytest run is a one-line summary with no stack trace, and the tail of a timeout is silence - the process was killed, so it printed nothing.

Why not patterns only? Because some real failures print no keyword at all. A cancelled job, a runner that vanished.

Why three lines of context? Because `E assert 19.99...` on its own does not tell you which file or which function. The line above it does.

Why the 60-line cap? Because a log that legitimately contains the word ERROR ten thousand times - a retry loop, a verbose linter - would otherwise blow the whole token budget. The cap makes the cost bounded regardless of input.

Why the omission markers? Without them the model sees two distant lines as adjacent and may invent a causal link between them. The marker tells it there is a gap.

The result. 12,017 lines to 40. 99.7% discarded. Roughly 240,000 tokens down to about 821 - a cost reduction of the same order.

And the risk, which the tool prints itself. The output says, in red: *this is the step most likely to throw away the answer*. That is a false negative. If the true cause was a setup step that failed quietly ten thousand lines earlier and printed no keyword, it is neither in the tail nor matched by a pattern. It gets dropped, and the model then explains the symptom with complete confidence, because from where it is standing the symptom is all there is.

On screen. Three boxes: the rule, the actual forty lines being sent, and the numbers.

SAY

> First box, the rule. Keep the last forty lines. Plus every line matching one of ten error patterns. Plus three lines of context around each match.
>
> Forty-three pattern matches in twelve thousand lines.
>
> Second box - and this is the part I want you to actually look at. That is not a summary. That is literally the text we send to the model. The red lines are the assertion failure. Everything else is context around it.
>
> Twelve thousand and seventeen lines, down to forty. Ninety-nine point seven percent thrown away.
>
> Now read that last red line. "This is the step most likely to throw away the

answer." We put that warning in our own output on purpose. If the real cause

was a setup step that failed quietly ten thousand lines earlier and printed

no error keyword, this filter drops it - and the model then explains the

wrong thing, very confidently. Quân comes back to that in the risks section.

If asked - "why not send the whole log, context windows are big now?" Two reasons. Cost is linear in input tokens, so it is a 300x difference per call. And more context measurably makes models worse at finding one specific fact. Trimming is an accuracy optimisation, not only a cost one.

Step 6 - `PromptBuilder.build()` *(0:10)*

What the code does. Assembles two things.

The **system prompt** is fixed: *you are a CI failure triage assistant, you are given the tail of a failed job log, base your answer on the actual error text rather than generic advice.* That last clause is doing real work - it is what pushes the model to quote `test_cart.py:87` instead of writing "check your test assertions."

The **user message** carries the repository, the run id, the commit SHA, then the trimmed log inside a fenced code block.

Why the metadata is included. It lets the model reference specifics, and it makes the resulting comment traceable back to a run.

Why the log is fenced. It marks where untrusted content begins and ends. It is a weak mitigation against prompt injection - a determined attacker can write a closing fence - but it costs nothing and it helps.

The thing to point out. The schema is *not* in this message. Under the prompt-only mechanism it would be, pasted in as text. Under tool use it travels in the `tools` field instead. That is precisely the difference the second demo measures.

SAY

Prompt assembled. Note what is *not* in it - the schema. The schema does not

go in the message. It goes in the tools field. That's the next step.

Step 7 - `ClaudeClient.complete()` - expanded *(0:35)*

What the request contains. Five things: `model`, `max_tokens`, `system`, `messages`, and the two that matter -

```
tools       = [ { name: "triage_result",
                  description: ...,
                  input_schema: <4 properties, 1 enum of 7> } ]
tool_choice = { "type": "tool", "name": "triage_result" }
```

`tool_choice` with type `"tool"` means the model **must** call that specific tool. Not *may*. There is no path where it replies with prose.

What comes back. The `content` field is a list of blocks. With forced tool use one of them has `type == "tool_use"`, and its `.input` is **already a Python dictionary**, validated against the JSON Schema on the server before it reaches us.

Compare with the other mechanism: there, `content[0].type == "text"` and `.text` is a string, and we run our own `extract_json()` over it - strip code fences, find the outermost braces, `json.loads`, hope.

The token numbers. About 1,877 input, about 200 output. Input tokens include the serialized tool definition, and that is where the roughly 28% overhead over prompt-only comes from. One call costs about a cent.

Why this is the only AI in the system. Eight of nine steps are ordinary engineering. This one step is the model. Everything around it exists to make its output safe to use.

On screen. The request shape as JSON, then the response block type, then the token counts.

SAY

> Request shape first. Model, max tokens, and then the two lines that matter -
>
> `tools`, with our schema, and `tool_choice` set to "tool", which forces the
>
> model to answer through the tool instead of writing prose.
>
> And the response. Content type: `tool_use`. Not `text`.
>
> `content[1].input` is already a Python dictionary.
>
> There is no JSON parsing anywhere in this path. That function does not exist
>
> here. Remember that in about ninety seconds.
>
> Eighteen hundred tokens in, two hundred out. About one cent.

If asked - "what if the model refuses or the API errors?" The call is wrapped. On any exception the demo prints the error and falls back to a saved response so the pipeline still completes. In production the behaviour is different and deliberate: log it, mark `comment_status = failed`, post nothing. A broken bot must never block or spam a pull request.

Step 8 - `ResponseValidator.validate()` *(0:10)*

Six rules, and the point is which ones the API already covers.

#	Rule	Guaranteed by the API?
1	all four required fields present	**yes**
2	no unexpected fields	**yes**
3	`failure_category` is one of the seven enum values	**yes**
4a	`confidence_score` is a number	**yes**
4b	`confidence_score` is within [0, 1]	**no** - we never set `minimum`/`maximum`
5	`root_cause` ≤ 600 characters	**no** - prose in a `description`
6	`suggested_fix` ≤ 600 characters	**no**

So half the validator is redundant and half is load-bearing. That is exactly the finding on slide 33: **structured output enforces shape, not meaning.**

One implementation detail worth knowing in case it comes up: the number check explicitly excludes booleans. In Python `isinstance(True, int)` is `True`, so `confidence_score: true` would pass a naive numeric check and then break formatting downstream. One extra clause.

SAY

> Six rules. All pass. And I'll come back in a minute to why this step still
>
> has to exist, even though the schema was enforced.

Step 9 - `MarkdownFormatter.format()` + `post_pr_comment()` *(0:10)*

Formatting. A fixed template: heading, category and confidence, root cause, suggested fix, then a footer. A fixed template is only possible *because* the output is structured - that is the payoff from steps 6 and 7 landing here.

Posting. `POST /repos/{owner}/{repo}/issues/{number}/comments`. Note `issues`, not `pulls` - on GitHub every pull request is also an issue for commenting purposes, and the `pulls` comment endpoint is for line-level review comments instead. This catches people out.

Permissions, and why they are the answer to a risk. The token carries exactly one permission: *Pull requests - read and write*. The bot cannot merge, cannot close, cannot re-run, cannot push. That is not an oversight, it is the mitigation for prompt injection through the log. If someone injects instructions into a test's output, the worst outcome is an embarrassing comment, not a compromised repository.

The footer. *Generated by Claude. Verify before acting.* On every comment, every time.

SAY

> Rendered. In the real bot this is a POST to the pull request. Here it writes
>
> the file.

Beat 10 - the output *(0:20)*

The JSON prints, then the rendered comment, then a one-line summary.

SAY

> That's the object. Four fields, the category from a seven-value enum, a
>
> confidence score.
>
> And that's what the developer sees on the pull request. Category, confidence,
>
> what broke - with the actual file and line number - and something to try.
>
> Twelve thousand lines in. One comment out. Under four seconds.
>
> Last line, every time: generated by Claude, verify before acting.

□ The move that proves it is live *(optional, 0:20 - only if you are ahead)*

Turn to the room **before** you run anything:

> Pick one. Dependency failure, syntax error, or infrastructure timeout.

Then run whichever they call out:

```
py triage_demo.py --log dependency
py triage_demo.py --log syntax
py triage_demo.py --log timeout
```

Five seconds. Different log, different category, different fix. This kills three suspicions at once: is it a recording, is it hard-coded, did it memorise one answer. If you have twenty spare seconds anywhere in the talk, spend them here.

PART C - The enforcement experiment *(1:30)*

Switch terminal. `cd ..\spike`

`py debug_trace.py --mode prompt-only` ***(0:45)***

SAY

> Same log, different question. This one is about *how* we force the shape.
>
> Mechanism A: the schema goes in the message, as text. We ask politely.
>
> Content type: `text`. That's a string. The model had no obligation to give us JSON - it chose to.
>
> Now our own function has to find the object inside that string. Strip the code fences, find the outermost braces, parse. Thirty lines that we wrote and we maintain.
>
> Today it worked.

`py debug_trace.py --mode tool-based` ***(0:45)***

SAY

> Mechanism B. Same log. Same model. Same schema.
>
> Content type: `tool_use`. Already a dictionary. That thirty-line function is not in this path at all.
>
> But look at the tokens. Fourteen sixty-five for A, eighteen seventy-seven for B. Twenty-eight percent more on every call, because the tool definition is sent as overhead.
>
> So B is safer and more expensive. Which do you pick? We ran it forty times to find out.

Then slide 31 - *It's a tie.*

PART D - Questions this demo invites

Answer in two sentences. Do not get defensive - every one of these has a good answer, and several of them earn marks.

"So you didn't actually build the webhook receiver?" Correct, not yet. The receiver is about twenty lines that parse a request and call `process_failed_run` - which is exactly what the demo calls directly. We spent the time on the nine steps behind it, because that is where the design risk was.

"Then how do you know the webhook path works?" We don't, and that is the honest gap. What we have not tested is GitHub's real delivery behaviour - retries, duplicate deliveries, out-of-order events. Our idempotency check on `run_id` is designed but not implemented. That is the biggest distance between this demo and a shipped bot.

"Isn't the log fake?" It is synthesized deterministically from a seed, and the failure at the end is a real pytest floating-point failure. We generate it rather than download it so the demo is reproducible - the same 12,017 lines every run, which is why the trimming ratio I quoted is a measurement and not a guess.

"Why not show a real GitHub Actions run?" It takes thirty seconds to several minutes and it can queue. And it would be testing GitHub's scheduler, not our code. Risk without information.

"How do we know the model isn't memorising that one log?" Pick a different one. *(then run `--log dependency` live)*

"What happens on a log format you didn't design for?" We don't know. We tested four shapes, all English, all GitHub Actions. A Jenkins log, a log in another language, or a log that itself contains JSON - those are exactly the cases we have not tried, and the trimmer's regexes are the part most likely to break.

"Is the API call real, or cached?" Real - you can see the token counts, and they shift slightly between runs because the wording changes. There is an `--offline` flag that uses a saved response and I am not using it. *(If you ARE using it because the network died, say so. Never present a cached run as live.)*

"Why is validation still needed if the schema is enforced?" That is the next slide. Short version: the schema enforces shape, not meaning. The six-hundred-character limit is English prose inside a description field, so nothing enforces it.

"What can this bot actually do to my repository?" Post a comment. That is the only permission the token has. It cannot merge, close, re-run or push. That is deliberate - it is our answer to prompt injection through the log.

"Where does the data go?" The trimmed log goes to Anthropic's API - forty lines, not the whole log. For a repository with secrets in the build output, redaction is a real problem we would have to solve first. We have not.

"How much does it cost to run?" About a cent per failure. That scales with your failure rate, not your commit rate. A busy monorepo with five hundred red runs a day is a real invoice, and that is on our risks slide.

PART E - Setup and failure handling

Before you stand up

- [ ] `py triage_demo.py --offline` runs clean - this is your floor, it cannot fail
- [ ] `py triage_demo.py --focus 5,7 --step` runs with a real API key
- [ ] Terminal font size **20**, window nearly full screen
- [ ] Command pre-typed, cursor waiting. **Never type in front of an audience.**
- [ ] Second window already at `..\spike` with `debug_trace.py` typed
- [ ] Screen recording of both demos saved to the Desktop
- [ ] Screenshots of both on hidden backup slides
- [ ] You know the four lines below by heart

If it breaks on stage

Say **one sentence**, then move. Never debug in front of the room.

What happened	Say this	Then
No network or API error	"Network's blocked here - this is the recorded run."	play the clip
Any other error	"Let me run the offline path."	`Ctrl-C`, then `py triage_demo.py --offline`
Colours wrong on the projector	*(say nothing)*	`--offline --no-color`
Hangs longer than ten seconds	"Slow link - here's the recording."	switch to the clip

The script already falls back to a saved response by itself if the API call throws, so it will not die mid-demo. Your real risk is not the code. It is standing in silence - ten seconds of nothing feels like a minute from the front of a room.

One thing you must not do. If you end up on the offline path, do not let the room believe it was live. One clause is enough - "this is the saved run" - and you keep every mark for honesty that the rest of this talk earns.